

hwk1/regsh2.vhd

```

1  -----
2  --
3  --  SH-2 Register Array
4  --
5  --  This is an implementation of a Register Array for the SH-2 it uses the
6  --  generic register array. Registers are accessed as single words, so
7  --  pair wise access is ignored. The SH-2 has 16 32-bit registers. For
8  --  addressing modes one of the bases is always R0, as such this is, thus, always
9  --  outputted. The other interface is used for the ALU and allows for output
10 --  simultaneously for data access and the ALU, for future pipelining.
11 --
12 --  Entities included are:
13 --      RegArraySH2 - the register array
14 --
15 --  Revision History:
16 --      6 Apr 26  Simone Shevchuk      Initial revision.
17 --      8 Apr 26  Simone Shevchuk      Declare default values
18 --      15 Apr 26 Simone Shevchuk      Cleanup deprecated signals, add comments
19 --      11 May 26 Simone Shevchuk      Change pair access inputs to all open
20 -----
21
22
23 --
24 --  RegArraySH2
25 --
26 --  This is a register array for SH2. The SH-2 has 16 32-bit registers through
27 --  the generic regarray implementation. This generates appropriate signals
28 --  from the control signals and sets excess signals like double word ones
29 --  appropriately to be ignored. There is also two separate access ports and
30 --  a write port to allow the registers to be used as addressable registers
31 --  simultaneous to their use in other blocks such as the ALU.
32 --
33 --
34 --  Inputs:
35 --      reg_in      - input bus to the register writes
36 --      reg_in_sel   - which register to write
37 --      reg_store    - actually write to a register
38 --      reg_a_sel    - register to read onto bus a for ALU
39 --      reg_b_sel    - register to read onto bus b for ALU
40 --      reg_ax_in    - input bus for address register updates
41 --      reg_ax_in_sel - which address register to write
42 --      reg_ax_store  - actually write to an address register
43 --      reg_a2_sel    - register to read onto address bus 2
44 --      clock        - the system clock
45 --
46 --  Outputs:
47 --      reg_a        - register value for bus a
48 --      reg_b        - register value for bus b
49 --      reg_a1       - register value for address bus 1, R0
50 --      reg_a2       - register value for address bus 2
51 --
52
53 library ieee;
54 use ieee.std_logic_1164.all;
55
56 entity RegArrSH2 is
57
58     port(
59         reg_in      : in  std_logic_vector(31 downto 0);
60         reg_in_sel   : in  integer range 15 downto 0;
61         reg_store    : in  std_logic;
62         reg_a_sel    : in  integer range 15 downto 0;
63         reg_b_sel    : in  integer range 15 downto 0;
64         reg_ax_in    : in  std_logic_vector(31 downto 0);
65         reg_ax_sel   : in  integer range 15 downto 0;

```

```

66     reg_ax_store    : in  std_logic;
67     reg_a2_sel      : in  integer range 15 downto 0;
68     clock           : in  std_logic;
69     reg_a           : out std_logic_vector(31 downto 0);
70     reg_b           : out std_logic_vector(31 downto 0);
71     reg_a1          : out std_logic_vector(31 downto 0);
72     reg_a2          : out std_logic_vector(31 downto 0)
73 );
74
75 end RegArrSH2;
76
77 architecture behavioral of RegArrSH2 is
78
79     signal reg_a1_sel : integer := 0; -- set to 0 to always read R0 for address bus 1
80
81 begin
82     -- instantiate generic register array with appropriate signal mapping
83     Regs : entity work.RegArray
84     generic map (
85         regcnt => 16,    -- 16 registers
86         wordsize => 32   -- 32-bit wide
87     )
88     port map (
89         RegIn      => reg_in,
90         RegInSel   => reg_in_sel,
91         RegStore   => reg_store,
92         RegASel    => reg_a_sel,
93         RegBSel    => reg_b_sel,
94         RegAxIn    => reg_ax_in,
95         RegAxInSel => reg_ax_sel,
96         RegAxStore => reg_ax_store,
97         RegA1Sel   => reg_a1_sel,
98         RegA2Sel   => reg_a2_sel,
99         RegDIn     => open,
100        RegDInSel  => open,
101        RegDStore  => open,
102        RegDSel    => open,
103        clock      => clock,
104        RegA       => reg_a,
105        RegB       => reg_b,
106        RegA1      => reg_a1,
107        RegA2      => reg_a2,
108        RegD       => open
109    );
110
111 end behavioral;
112

```